CalculatorTest — Implementation Notes

Renaissance Global programming challenge | .NET 8 (C#) back-end + Angular front-end

What I built

A small calculator implemented as two cooperating parts. The **back-end** (C# / .NET 8) holds the maths logic behind the supplied `ISimpleCalculator` interface, is covered by unit tests, and is exposed through a lightweight web service. The **front-end** (Angular) provides a responsive UI that performs the operations inside a modal pop-up with a header and footer, and lets the user restyle that pop-up live. All nine requirements are covered.

Solution structure

`CalculatorTest.sln`	Solution grouping the three C# projects (requirement 1).
`src/Calculator.Core`	Class library: the ISimpleCalculator interface + SimpleCalculator implementation (req. 2-3).
`src/Calculator.Api`	Minimal-API web service exposing add/subtract (req. 5).
`tests/Calculator.Tests`	xUnit unit tests for the calculator (req. 4).
`calculator-web`	Angular app: modal calculator, responsive layout, live theming (req. 6-9).

Key technical decisions

Minimal APIs instead of controllers. The service exposes only two simple endpoints (add and subtract), so I declared them directly in `Program.cs` using the minimal-API style (`app.MapGet(...)`) rather than adding MVC controller classes. For a surface this small, minimal APIs are the lighter, more readable choice and are the current recommended approach in .NET 8. Each endpoint still does exactly what a controller action would; it is simply declared inline. I would move to controllers if the endpoint count or routing complexity grew.

Programming to the interface (dependency injection). The implementation is registered as `ISimpleCalculator` → `SimpleCalculator` in the DI container, and the endpoints and tests depend on the interface, not the concrete class. This keeps the design loosely coupled and makes the calculator easy to swap or mock.

Overflow handled explicitly. Because the interface uses `int`, I perform the arithmetic in a `checked` context so that exceeding the 32-bit range raises a clear error rather than silently wrapping to a wrong value. The web service catches this and returns a 400 response; a unit test verifies the behaviour.

CORS for the Angular client. The API allows the Angular dev server (`http://localhost:4200`) so the browser can call it during local development. Swagger UI is enabled for quick manual testing.

How I developed and ran it

I built this on **macOS using Visual Studio Code** (not Visual Studio), driving the .NET side entirely with the **dotnet CLI** and the Angular side with the **Angular CLI / npm**. Because everything targets .NET 8 (cross-platform) and standard Angular, the solution opens and runs unchanged on Windows with Visual Studio 2022 as well — the `.sln` opens all projects, tests run in Test Explorer, and the API runs with F5. The Angular app runs the same way on any OS via npm.

```
# back-end
dotnet build CalculatorTest.sln
dotnet test
dotnet run --project src/Calculator.Api  # http://localhost:5000

# front-end
cd calculator-web
npm install
npm start  # http://localhost:4200
```

Note: the C# service must be running for the calculator buttons to work, since the front-end sends each calculation to it.